

创建表时:

PK:主键	NN: 不能为空	UQ: 代表唯一	AI: 自动增长
-------	----------	----------	----------



SQL分类

- (1)DDL:全称Data Definition Language,是数据定义语言,用来定义数据库对象:库、表、列等.
- (2)DML:全称Data Manipulation Language,是数据操作语言, 用来定义数据库记录（数据）.
- (3)DCL:全称Data Control Language,是数据控制语言, 用来定义访问权限和安全级别.
- (4)DQL:全称Data Query Language,是数据查询语言, 用来查询记录（数据）.

int:	整型
double:	浮点型, 例如double(5,2)表示最多5位, 其中必须有2位小数, 即最大值为999.99;
char:	固定长度字符串类型; (当输入的字符不够长度时会补空格,所以在查询信息的时候可能存在查找不到的情况,一般不用)
varchar:	固定长度字符串类型(当输入的字符不够长度时不会补空格,推荐使用);
text:	字符串类型;
date:	日期类型
time:	时间类型

Review SQL Script

Apply SQL Script

Review the SQL Script to be Applied on the Database

Online DDL

Algorithm:

Default

Lock Type:

Default

```
1 CREATE TABLE 'sql'. 'student' (  
2   'Sno' CHAR(3) NOT NULL,  
3   'Sname' CHAR(8) NOT NULL,  
4   'Ssex' CHAR(2) NOT NULL,  
5   'Sbirthday' DATETIME NULL,  
6   'Class' CHAR(5) NULL,  
7   PRIMARY KEY ('Sno'));  
8
```

Review SQL Script

Apply SQL Script

Review the SQL Script to be Applied on the Database

Online DDL

Algorithm:

Default

Lock Type:

Default

```
1 CREATE TABLE 'sql'. 'score' (  
2   'Sno' CHAR(3) NOT NULL,  
3   'cno' CHAR(5) NOT NULL,  
4   'Degree' DECIMAL NULL,  
5   INDEX 'score_idx' ('Sno' ASC) VISIBLE,  
6   INDEX 'score_idx1' ('cno' ASC) VISIBLE,  
7   CONSTRAINT 'score'  
8     FOREIGN KEY ('Sno')  
9     REFERENCES 'sql'. 'student' ('Sno')  
10    ON DELETE NO ACTION  
11    ON UPDATE NO ACTION,  
12  CONSTRAINT 'score'  
13    FOREIGN KEY ('cno')  
14    REFERENCES 'sql'. 'course' ('Cno')  
15    ON DELETE NO ACTION  
16    ON UPDATE NO ACTION);  
17
```

vt(sno),

z(cno),

Back

Apply

Cancel

增删改查

2023年3月28日 14:38

打开数据库语法: use 数据库名 (服务器内的名称)

例: 一个数据库为名为: student 则: Use student

Desc 打印表结构

数据库的创建:

```
1 create table stuinfo --创建学生信息stuinfo表
2 (
3     --创建列开始
4     sid int primary key, --sid 学生编号 类型int 主键
5     sname nchar(8) not null, --sname 学生姓名 类型nchar(8) 非空
6     saddress nvarchar(30), --saddress 学生地址 类型nvarchar(30)
7     sclass int, --sclass 学生班级 类型int
8     ssex nchar(1) --ssex 学生性别 类型nchar(1)
9 )
```

插入数据:

语法一: insert into 表名 (列名一, 列名二, 列名三) values (数据一,数据二,数据三)

例: insert into stuinfo(sid,sname,saddress,sclass,ssex)values(1,'张三','地球', 1001,'男')

结果

消息

执行计划

	sid	sname	saddress	sclass	ssex
1	1	码仙1	火星	1001	男

https://blog.csdn.net/tswc_byy

语法二: insert into 表名 values (数据1, 数据2, 数据3, 数据4)

例:insert into stuinfo values(2,'张三2', '地球')

结果

消息

执行计划

	sid	sname	saddress	sclass	ssex
1	1	码仙1	火星	1001	男
2	2	码仙2	火星	1002	女

https://blog.csdn.net/tswc_byy

语法三: (插入部分数据) insert into 表名(列名1, 列名2) values (数据1, 数据2)

使用限制: 主键和非空约束列必须添加数据

Insert into stuinfo(sid,sname,sclass) values (3,'张三3', 1003)

	sid	sname	saddress	sclass	ssex
1	1	码仙1	火星	1001	男
2	2	码仙2	火星	1002	女
3	3	码仙3	NULL	1003	NULL

查询:

- 1、查询单条数据(也可以说查询一行数据)

语法: select * from 表名 where 查询条件

Select * from stuinfo where sid=2

	sid	sname	saddress	sclass	ssex
1	2	码仙2	火星	1002	女

- 2、查询整张表

语法: select * from 表名

Select * from stuinfo

	sid	sname	saddress	sclass	ssex
1	1	码仙1	火星	1001	男
2	2	码仙2	火星	1002	女
3	3	码仙3	NULL	1003	NULL

修改表的数据:

- 1、修改一个数据

语法: update 表名 set 列名=新数据 where 查询条件

Update stuinfo set saddress='木星' where sid=1

	sid	sname	saddress	sclass	ssex
1	1	码仙1	木星	1001	男
2	2	码仙2	火星	1002	女
3	3	码仙3	NULL	1003	NULL

- 2、修改一行数据

语法: update 表名 set 列名1=新数据1, 列名2=新数据2, 列名3=新数据3, where 查询条件

Update stuinfo set sname='码仙4', saddress='木星',sclass=4,ssex='女' where sid=1;

	sid	sname	saddress	sclass	ssex
1	1	码仙4	木星	4	女
2	2	码仙2	火星	1002	女
3	3	码仙3	NULL	1003	NULL

数据表中数据的删除:

语法: delete from 表名 where 查询条件;

Delete from stuinfo where sid =2

	sid	sname	saddress	sclass	ssex
1	1	码仙4	木星	4	女
2	3	码仙3	NULL	1003	NULL

添加列: alter table 表名 add 列名 varchar(55)

删除列: alter table 表名 drop column 列名

改列类型: alter table 表名 alter column 列名 varchar(22)

修改列名称: exec sp_rename "表名.字段名", '新名', 'column'

修改表名称: alter table 旧表名 rename as 新表名, or rename table 旧表名 to 新表名;

给字段添加主键: alter table 表名 add primary key (字段名)

给字段添加唯一约束: alter table 表名 modify 字段名 字段类型 UNIQUE

删除字段唯一约束: ALTER TABLE 表名 DROP INDEX 字段名

主键And外键

2023年4月11日 16:06

主键和唯一的对比

	保证唯一性	是否允许为空	一个表中可以有几个 最多有一个 可以有多个	是否允许组合 √ √ 不推荐
主键	√	×		
唯一	√	√		

COMMENT = '表的注释'

外键:

1. 在从表上设计**外键**关系, 来限制从表的**取值**。主表: 要引用的表, 从表是: 指向主表的表。
2. 从表的**外键**列类型和主表的关联列一样兼容
3. 主表的关联列**必须是一个主键或唯一**
4. 插入表的时候**先**插入主表, **再**插入从表
删除数据时, 删除从表, 再删除 主表。

外键参数:

On delete |update [no action] [set null] [set default] [cascade]

No action 不做任何操作

Set null 表示在外键中将相应字段设置为null

Set default 表示设置为默认值

Cascade 表示级联操作

1. 删除主键约束
ALTER TABLE 表名 DROP primart key;
2. 删除唯一约束|索引
ALTER TABLE 表名 DROP index 约束名|索引名
DROP index 约束名 | 索引名 on 表名

识别并定义表的自动增长列

Auto increment

定义为auto_increen 属性的字段, 必须是整数类型, 而且必须是一个主键或唯一键

auto_increment 的初始值可以在表后面的属性中使用 "auto_increment=初始值" 来

定义，否则初始值默认从1开始。

```
create table mojour(id int auto_increment unique,mname char(8) not null) auto_increment = 5;
```

区分大小写: BINARY

Name char(8), BINARY

数据库设计规范化

2023年5月30日 17:16

第一范式的目标是确保每列的原子性

如果每列都是不可再分的最小数据单元（也称为最小的原子单元），则满足第一范式
也就是说表里的字段，它的应用范围应该是最小的范围。比如一段地址它拥有国家、省、市、区、乡镇、详细地址，表中的地址不能只有一个字段，该字段里包含所有信息，而应该将国家、省、市、区、乡镇、详细地址分别分类为几个字段了。

第二范式 (2nd NF)

如果一个关系满足1NF，并且出任何一个非主键字段的数值都依赖于主键字段，则满足第二范式

第二范式要求每个表只描述一件事情。

也就是说满足第二范式的前提就是满足第一范式，其次就是表的任何一个非主键的字段都只依赖于主键字段，不能出现表中的某个字段依赖于别的字段。

第三范式 (3rd NF)

若一个关系满足第一范式和第二范式，以及任何两个非主键字段的数值之间不存在函数依赖关系，则满足第三范式。

也就是说满足第三范式的前提就是满足第一和第二范式，以及表内不能有函数计算关系，就是不能在表内进行计算，如果要计算，只能在程序里把数据库的数据提取出来进行计算。

Orders	
字 段	例 子
订单编号	001
产品编号	AB001
<u>单价</u>	30
<u>数量</u>	50
<u>金额</u>	1500

内连接

2025年11月10日 16:53

连接查询-内连接

内连接查询语法:

➤ 隐式内连接

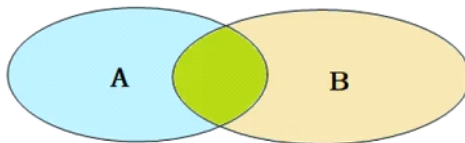
```
SELECT 字段列表 FROM 表1,表2 WHERE 条件 ...;
```

➤ 显式内连接

```
SELECT 字段列表 FROM 表1 [INNER] JOIN 表2 ON 连接条件 ...;
```

内连接查询的是两张表交集的部分

40. 基础-多表查询-外连接



隐式内链接

```
SELECT e.emp_name, d.dept_name FROM employees e, departments d WHERE e.dept_id = d.dept_id
```

显示内链接, 显示内连接中的INNER可以省略

```
SELECT e.emp_name, d.dept_name FROM employees e INNER JOIN departments d ON e.dept_id = d.dept_id
```

外连接

2025年11月10日 17:35

连接查询-外连接

外连接查询语法:

➤ 左外连接

```
SELECT 字段列表 FROM 表1 LEFT [OUTER] JOIN 表2 ON 条件 ...;
```

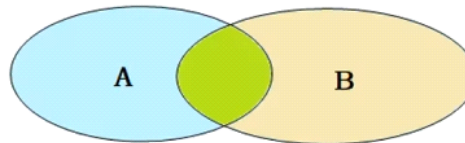
相当于查询表1(左表)的所有数据 包含 表1和表2交集部分的数据

➤ 右外连接

```
SELECT 字段列表 FROM 表1 RIGHT [OUTER] JOIN 表2 ON 条件 ...;
```

相当于查询表2(右表)的所有数据 包含 表1和表2交集部分的数据

播放41. 基础-多表查询-自连接



多表查询

2025年11月12日 15:03

联合查询-union , union all

对于union查询，就是把多次查询的结果合并起来，形成一个新的查询结果集。

```
SELECT 字段列表 FROM 表A ...  
UNION [ ALL ]  
SELECT 字段列表 FROM 表B ...;
```

对于联合查询的多张表的列数必须保持一致，字段类型也需要保持一致。

基本语法

2025年11月27日 14:36

- 介绍

视图（View）是一种虚拟存在的表。视图中的数据并不在数据库中实际存在，行和列数据来自定义视图的查询中使用的表，并且是在使用视图时动态生成的。

通俗的讲，视图只保存了查询的SQL逻辑，不保存查询结果。所以我们在创建视图的时候，主要的工作就落在创建这条SQL查询语句上。

- 作用

- 简单

视图不仅可以简化用户对数据的理解，也可以简化他们的操作。那些被经常使用的查询可以被定义为视图，从而使得用户不必为以后的操作每次指定全部的条件。

- 安全

数据库可以授权，但不能授权到数据库特定行和特定的列上。通过视图用户只能查询和修改他们所能见到的数据

- 数据独立

视图可帮助用户屏蔽真实表结构变化带来的影响。

视图

- 创建

```
CREATE [OR REPLACE] VIEW 视图名称(列名列表) AS SELECT语句 [WITH [ CASCADED | LOCAL ] CHECK OPTION ]
```

- 查询

```
查看创建视图语句: SHOW CREATE VIEW 视图名称;
查看视图数据: SELECT * FROM 视图名称 .....
```

- 修改

```
方式一: CREATE [OR REPLACE] VIEW 视图名称(列名列表) AS SELECT语句 [WITH [ CASCADED | LOCAL ] CHECK OPTION ]
方式二: ALTER VIEW 视图名称(列名列表) AS SELECT语句 [WITH [ CASCADED | LOCAL ] CHECK OPTION ]
```

- 删除

```
DROP VIEW [IF EXISTS] 视图名称 [,视图名称] ...
```

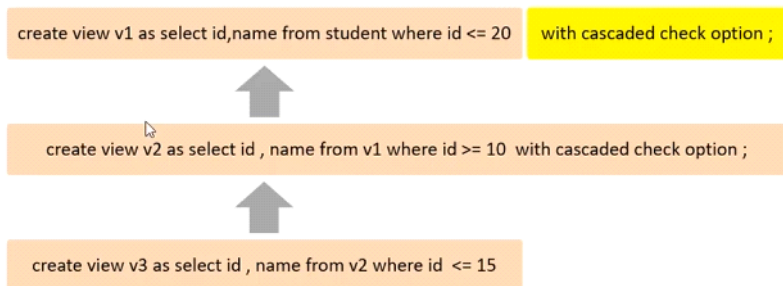
检查选项

2025年11月27日 14:40

● 视图的检查选项

当使用WITH CHECK OPTION子句创建视图时，MySQL会通过视图检查正在更改的每个行，例如 插入，更新，删除，以使其符合视图的定义。MySQL允许基于另一个视图创建视图，它还会检查依赖视图中的规则以保持一致性。为了确定检查的范围，mysql提供了两个选项：CASCADED 和 LOCAL，默认值为 CASCADED。

CASCADED：



如果在创建视图时没有添加检查选项则在操作视图时则不会检查条件，如在创建视图时约定的条件是表的某个范围id，没有添加视图则可以随意插入id为任何数的数据。如上图

当加了CASCADED时则会进行检查操作的语句不符合当前语句中WHERE的条件、如果当前视图是基于上一视图添加的，则还会向上检查是否满足上一视图的条件。如上图。

如果第一个视图添加了检查选项，第二个视图是基于第一个视图创建的新视图，而创建新视图中的语句没有添加检查选项在给新视图插入数据时则不会检查创建新视图的条件，但是会向上检查第一个视图的条件。

● 视图的检查选项

当使用WITH CHECK OPTION子句创建视图时，MySQL会通过视图检查正在更改的每个行，例如 插入，更新，删除，以使其符合视图的定义。MySQL允许基于另一个视图创建视图，它还会检查依赖视图中的规则以保持一致性。为了确定检查的范围，mysql提供了两个选项：CASCADED 和 LOCAL，默认值为 CASCADED。

```

-- local
create or replace view stu_v_4 as select id,name from student where id <= 15 ;

insert into stu_v_4 values(5,'Tom');

insert into stu_v_4 values(16,'Tom');

create or replace view stu_v_5 as select id,name from stu_v_4 where id >= 10 with local check option ;

insert into stu_v_5 values(13,'Tom');

insert into stu_v_5 values(17,'Tom');

create or replace view stu_v_6 as select id,name from stu_v_5 where id < 20 ;

insert into stu_v_6 values(14,'Tom');

```

LOCAL属性的作用是当本条语句添加了检查之后会检查当前的条件，如果当前视图有依赖还会检查所依赖的视图有无添加条件，如果有则检查，没有则不检查。

概述

2025年12月1日 20:05

索引概述

● 优缺点

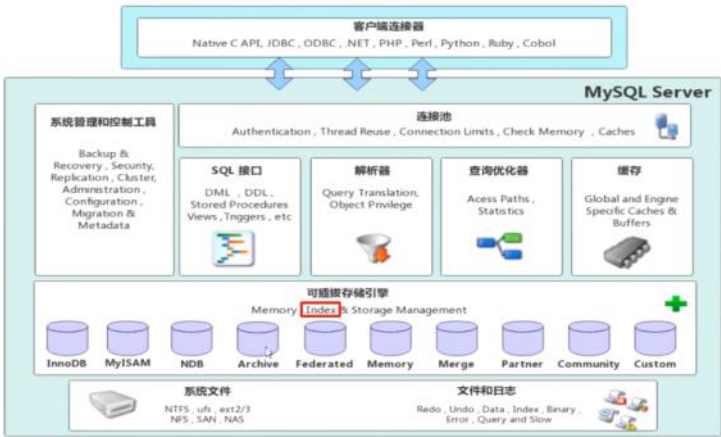
优势	劣势
提高数据检索的效率，降低数据库的IO成本	索引列也是要占用空间的。
通过索引列对数据进行排序，降低数据排序的成本，降低CPU的消耗。	索引大大提高了查询效率，同时却也降低更新表的速度，如对表进行INSERT、UPDATE、DELETE时，效率降低。

显而易见，索引的优点就是能提高查询效率降低消耗，但是劣势就是索引列也是需要占用空间和索引会降低操作表的降低，因为每次操作表都要维护索引。

结构

2025年12月1日 20:13

索引结构



索引结构

MySQL的索引是在存储引擎层实现的，不同的存储引擎有不同的结构，主要包含以下几种：

索引结构	描述
B+Tree索引	最常见的索引类型，大部分引擎都支持 B+ 树索引
Hash索引	底层数据结构是用哈希表实现的, 只有精确匹配索引列的查询才有效, 不支持范围查询
R-tree(空间索引)	空间索引是MyISAM引擎的一个特殊索引类型，主要用于地理空间数据类型，通常使用较少
Full-text(全文索引)	是一种通过建立倒排索引,快速匹配文档的方式。类似于Lucene,Solr,ES

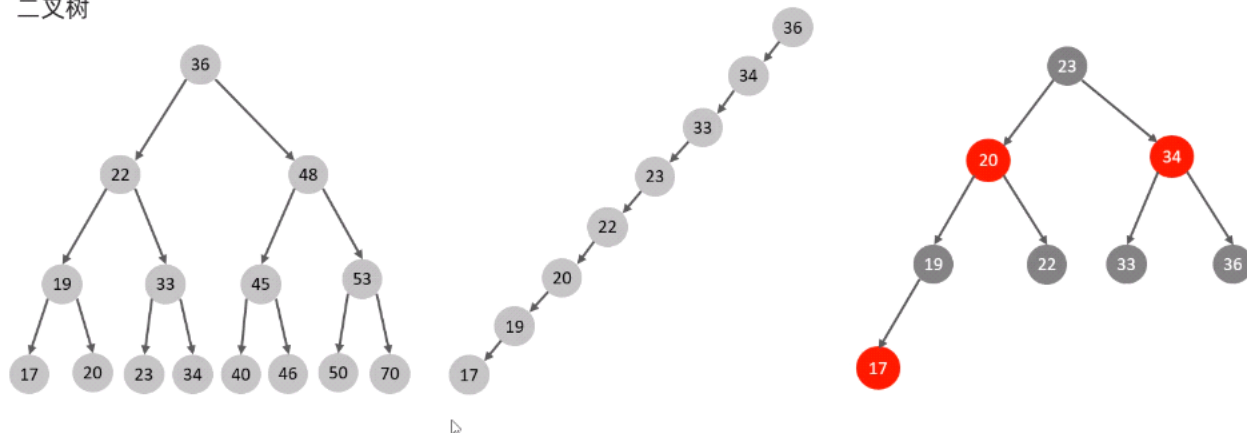
索引结构

索引	InnoDB	MyISAM	Memory
B+tree索引	支持	支持	支持
Hash 索引	不支持	不支持	支持
R-tree 索引	不支持	支持	不支持
Full-text	5.6版本之后支持	支持	不支持

我们平常所说的索引，如果没有特别指明，都是指B+树结构组织的索引。

索引结构

● 二叉树

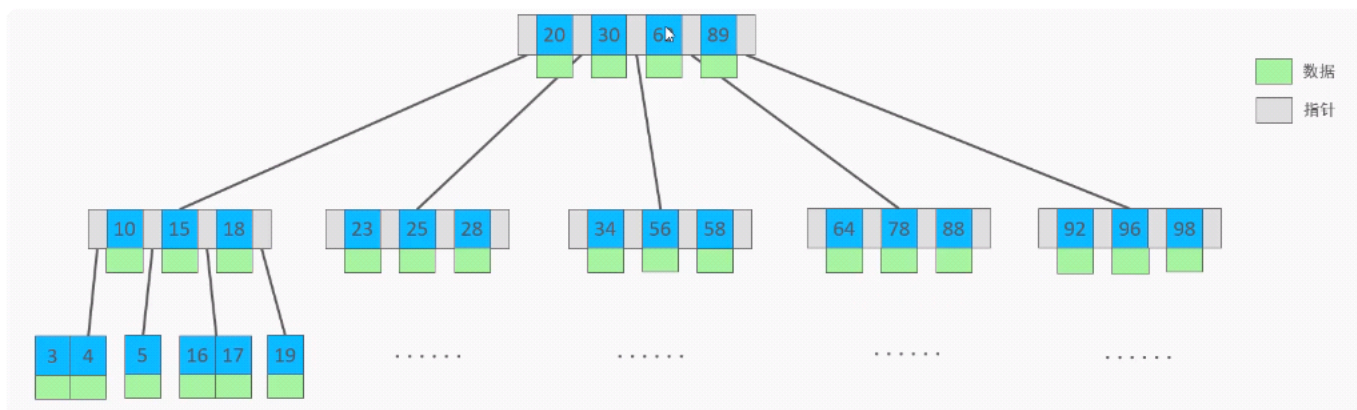


二叉树缺点：顺序插入时，会形成一个链表，查询性能大大降低。大数据量情况下，层级较深，检索速度慢。

红黑树：大数据量情况下，层级较深，检索速度慢。

● B-Tree（多路平衡查找树）

以一颗最大度数（max-degree）为5(5阶)的b-tree为例(每个节点最多存储4个key，5个指针)：



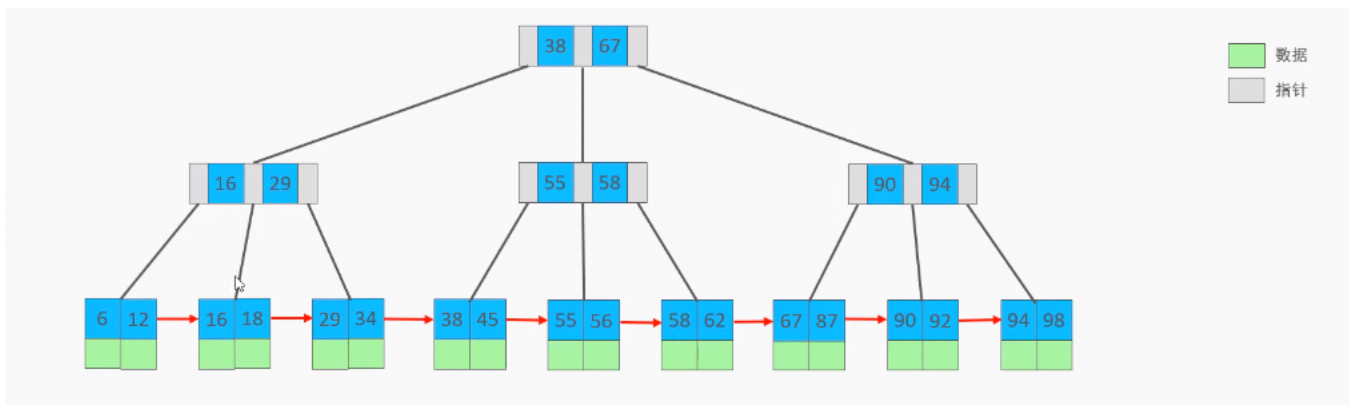
知识小贴士：树的度数指的是一个节点的子节点个数。

B树就是优化了二叉树的缺点，B树会以一颗最大度数的节点进行分裂，如图，这里的最大度数是5，又叫5阶，当一个节点有4个key时又有了五个指针时就会进行分裂，一个B树的最大指针不超过数的阶数，图中顶部的节点是这样理解的，小于二十的指针、二十到三十之间的指针、三十到六十二之间的指针、六十二到八十九之间的指针、大于八十九的指针，所以有五个指针。

进行分裂会按照取中间元素向上分裂的原则，例如顶部节点又插入一个元素90，此时的中间元素为62，62则会向上成为一个新的节点。

● B+Tree

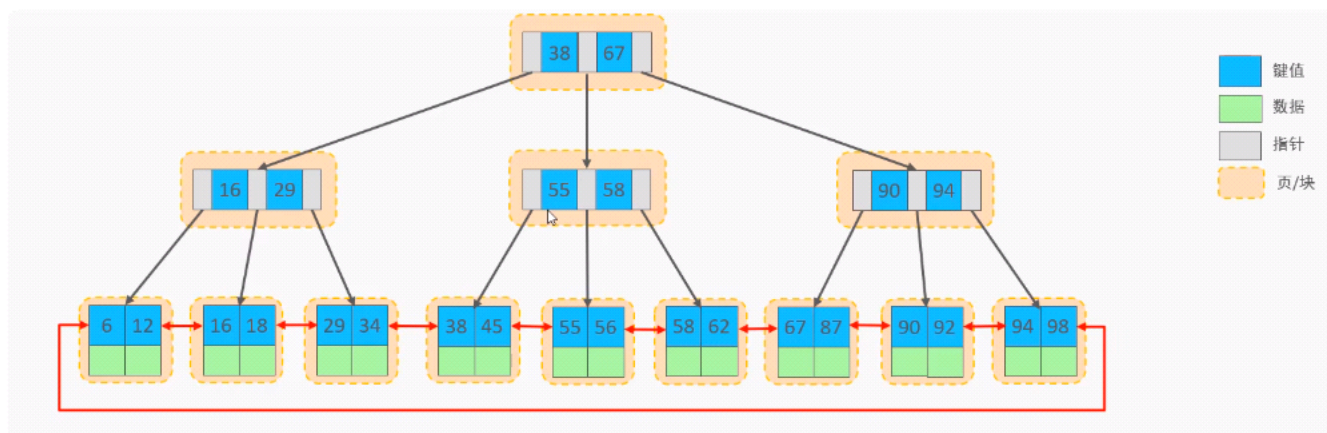
以一颗最大度数（max-degree）为4（4阶）的b+tree为例：



B+树与B树相同的是B+树也会向上分裂节点，但是在子节点中会保留分裂的树且只有子节点为数据，其余为指针。

● B+Tree

MySQL索引数据结构对经典的B+Tree进行了优化。在原B+Tree的基础上，增加一个指向相邻叶子节点的链表指针，就形成了带有顺序指针的B+Tree，提高区间访问的性能。

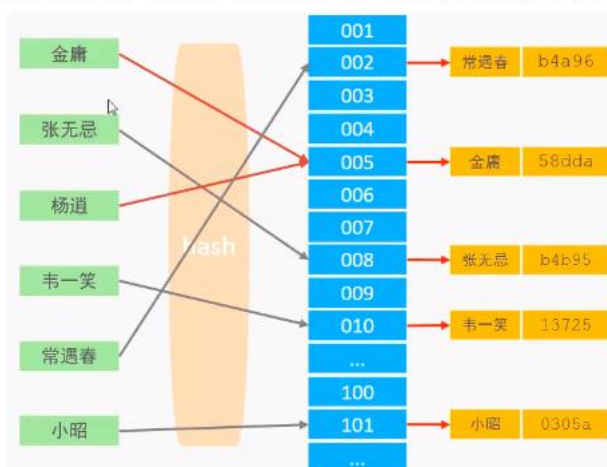


● Hash

哈希索引就是采用一定的hash算法，将键值换算成新的hash值，映射到对应的槽位上，然后存储在hash表中。

如果两个(或多个)键值，映射到一个相同的槽位上，他们就产生了hash冲突（也称为hash碰撞），可以通过链表来解决。

	id	name	age
58dda	1	金庸	36
b4b95	2	张无忌	22
a00ac	3	杨逍	33
13725	4	韦一笑	48
b4a96	5	常遇春	53
0305a	6	小昭	19
48c00	7	灭绝	45
f2d22	8	周芷若	17
e29fd	9	丁敏君	23
a7b7d	10	赵敏	28
4af6d	11	鹿杖客	49
00a22	12	鹤笔翁	68



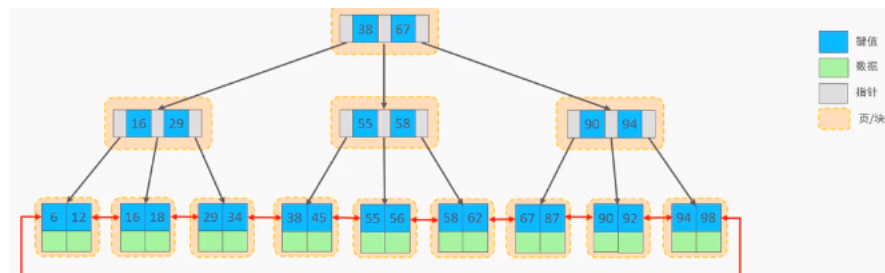
● Hash

➤ Hash索引特点

1. Hash索引只能用于对等比较(=, in), 不支持范围查询 (between, >, <, ...)
2. 无法利用索引完成排序操作
3. 查询效率高, 通常只需要一次检索就可以了, 效率通常要高于B+tree索引

➤ 存储引擎支持

在MySQL中, 支持hash索引的是Memory引擎, 而InnoDB中具有自适应hash功能, hash索引是存储引擎根据B+Tree索引在指定条件下自动构建的。



为什么InnoDB存储引擎选择使用B+tree索引结构?

- 相对于二叉树, 层级更少, 搜索效率高;
- 对于B-tree, 无论是叶子节点还是非叶子节点, 都会保存数据, 这样导致一页中存储的键值减少, 指针跟着减少, 要同样保存大量数据, 只能增加树的高度, 导致性能降低;
- 相对Hash索引, B+tree支持范围匹配及排序操作;

分类

2025年12月1日 21:12

分类	含义	特点	关键字
主键索引	针对于表中主键创建的索引	默认自动创建, 只能有一个	PRIMARY
唯一索引	避免同一个表中某数据列中的值重复	可以有多个	UNIQUE
常规索引	快速定位特定数据	可以有多个	
全文索引	全文索引查找的是文本中的关键词, 而不是比较索引中的值	可以有多个	FULLTEXT

在InnoDB存储引擎中，根据索引的存储形式，又可以分为以下两种：

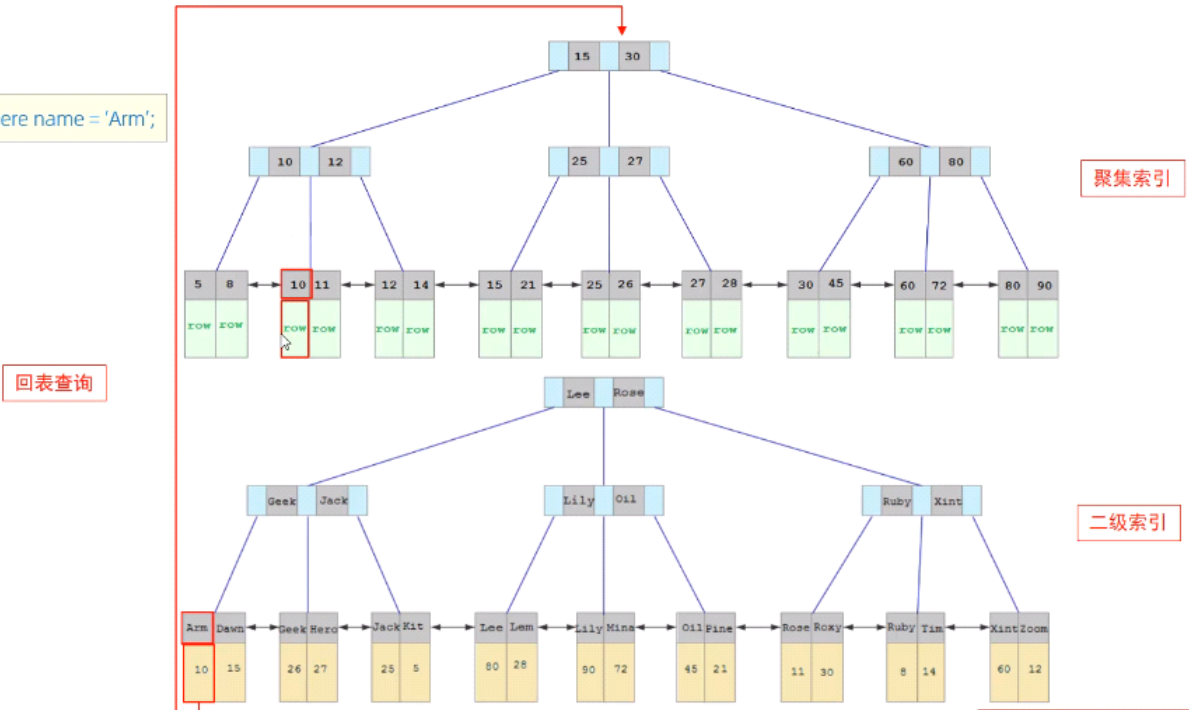
分类	含义	特点
聚集索引(Clustered Index)	将数据存储与索引放到了一块，索引结构的叶子节点保存了行数据	必须有,而且只有一个
二级索引(Secondary Index)	将数据与索引分开存储，索引结构的叶子节点关联的是对应的主键	可以存在多个

聚集索引选取规则：

- 如果存在主键，主键索引就是聚集索引。
- 如果不存在主键，将使用第一个唯一（UNIQUE）索引作为聚集索引。
- 如果表没有主键，或没有合适的唯一索引，则InnoDB会自动生成一个rowid作为隐藏的聚集索引。

索引分类

`select * from user where name = 'Arm';`



聚集索引拿到的是叶子节点的行数据，二级索引拿到的与叶子节点关联的主键
例如图查询的是Arm，该查询不是聚集索引而是二级索引，所以先查叶子节点中对应的主键然

后回到聚集索引中查询叶子节点的行数据区，而这个过程有一个专业名词叫做回表查询。

语法

2025年12月1日 21:32

- 创建索引

```
CREATE [ UNIQUE | FULLTEXT ] INDEX index_name ON table_name ( index_col_name,... );
```

- 查看索引

```
SHOW INDEX FROM table_name ;
```

- 删除索引

```
DROP INDEX index_name ON table_name ;
```

创建索引中红色框代表索引分类，蓝色框代表索引名称，紫色框代表关联的字段，如果只指定一个字段则称为单列索引，如果关联多个字段称为联合索引或组合索引

```
CREATE [ UNIQUE | FULLTEXT ] INDEX index_name ON table_name ( index_col_name,... );
```

事务操作

2025年12月9日 19:31

事务操作

- 查看/设置事务提交方式

```
SELECT @@autocommit;  
SET @@autocommit = 0;
```

- 提交事务

```
COMMIT;
```

- 回滚事务

```
ROLLBACK;
```

SELECT @@autocommit用于查看事务提交方式，为1是自动提交、0为手动提交
SET @@autocommit 用于设置事务提交方式

- 开启事务

```
START TRANSACTION 或 BEGIN;
```

- 提交事务

```
COMMIT;
```

- 回滚事务

```
ROLLBACK;
```

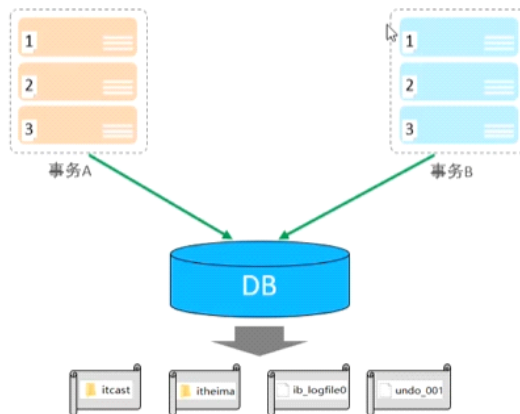
如果要显示的开启事务可以使用图中的命令来操作，如

```
start TRANSACTION  
select * from account where name = '张三';  
UPDATE account set money = money - 1000 where name = '张三';  
程序报错  
UPDATE account set money = money + 1000 where name = '李四';  
commit;  
rollback
```

四大特性

2025年12月9日 19:51

- 原子性 (Atomicity)：事务是不可分割的最小操作单元，要么全部成功，要么全部失败。
- 一致性 (Consistency)：事务完成时，必须使所有的数据都保持一致状态。
- 隔离性 (Isolation)：数据库系统提供的隔离机制，保证事务在不受外部并发操作影响的独立环境下运行。
- 持久性 (Durability)：事务一旦提交或回滚，它对数据库中的数据的改变就是永久的。



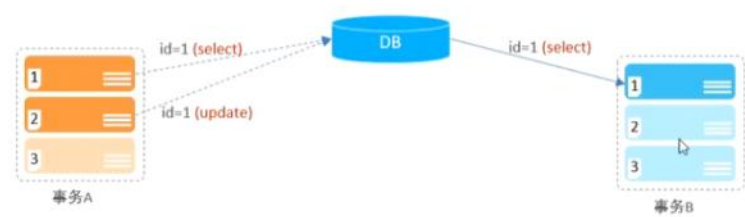
原子性就是

数据库系统

并发事务问题

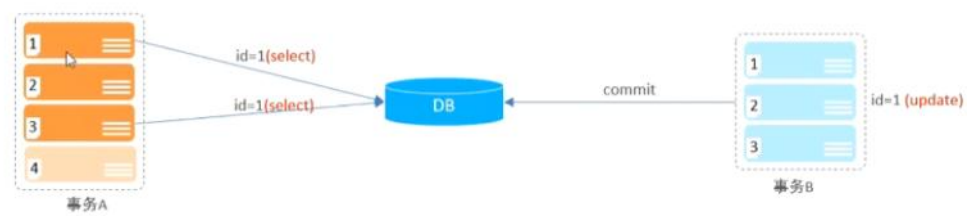
2025年12月9日 19:56

问题	描述
脏读	一个事务读到另外一个事务还没有提交的数据。
不可重复读	一个事务先后读取同一条记录，但两次读取的数据不同，称之为不可重复读。
幻读	一个事务按照条件查询数据时，没有对应的数据行，但是在插入数据时，又发现这行数据已经存在，好像出现了“幻影”。

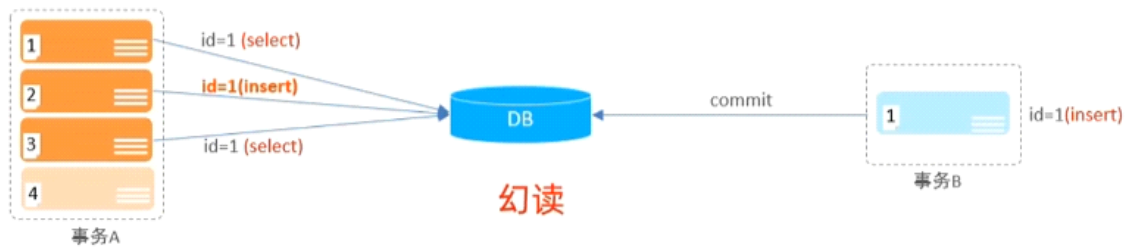


脏读：如图，当事务A中的查询id为1的语句在执行时发现数据库里并没有该数据，随后执行了第二条插入数据的语句，这时事务B执行了查询id为1的语句读取到了刚才插入的数据，但是事务A还没有提交，这时就是脏读

问题	描述
脏读	一个事务读到另外一个事务还没有提交的数据。
不可重复读	一个事务先后读取同一条记录，但两次读取的数据不同，称之为不可重复读。
幻读	一个事务按照条件查询数据时，没有对应的数据行，但是在插入数据时，又发现这行数据已经存在，好像出现了“幻影”。



不可重复读：事务A在查询数据库中id为1的数据，但是此时并没有id为1的数据，此时并发来了事务B对id为1的进行了插入数据并提交了事务，然后事务A在后面又执行了查询id为1的语句，此时却查询到了数据，这就是不可重复读



幻读：事务A的第一条语句在查询数据库中id为1的数据，此时数据库并没有数据，这时候事务B提交了给id为1插入数据的事务，然后事务A第二条语句也是一条给id为1的数据的语句，但由于id是主键，不能重复插入数据所以提示已经有了数据，这时来到了第三条查询id为1的语句，这里前提是解决了不可重复读的问题，查询id为1的语句还是没有，这种现象就称为幻读。

注意：事务隔离级别越高，数据越安全，但是性能越低。

事务隔离级别

2025年12月9日 20:20

隔离级别	脏读	不可重复读	幻读
Read uncommitted	√	√	√
Read committed	×	√	√
Repeatable Read(默认)	×	×	√
Serializable	×	×	×

```
-- 查看事务隔离级别
SELECT @@TRANSACTION_ISOLATION;

-- 设置事务隔离级别
SET [ SESSION|GLOBAL ] TRANSACTION ISOLATION LEVEL { READ UNCOMMITTED | READ COMMITTED | REPEATABLE READ | SERIALIZABLE }
```

session表示当前会话级别、global表示全局级别

分类

2025年12月9日 20:55

● 分类

MySQL中的锁，按照锁的粒度分，分为以下三类：

1. 全局锁：锁定数据库中的所有表。
2. 表级锁：每次操作锁住整张表。
3. 行级锁：每次操作锁住对应的行数据。

基本语法

2025年12月17日 15:33

● 创建

```
CREATE PROCEDURE 存储过程名称 ([参数列表])  
BEGIN  
    -- SQL语句  
END ;
```

● 调用

```
CALL 名称 ([参数]);
```

● 查看

```
SELECT * FROM INFORMATION_SCHEMA.ROUTINES WHERE ROUTINE_SCHEMA = 'xxx'; -- 查询指定数据库的存储过程及状态信息  
SHOW CREATE PROCEDURE 存储过程名称; -- 查询某个存储过程的定义
```

● 删除

```
DROP PROCEDURE [ IF EXISTS ] 存储过程名称 ;
```

注意: 在命令行中, 执行创建存储过程的SQL时, 需要通过关键字 `delimiter` 指定SQL语句的结束符。

变量

2025年12月17日 15:51

- 变量

系统变量 是MySQL服务器提供，不是用户定义的，属于服务器层面。分为全局变量（**GLOBAL**）、会话变量（**SESSION**）。

- 查看系统变量

```
SHOW [ SESSION | GLOBAL ] VARIABLES ;           -- 查看所有系统变量
SHOW [ SESSION | GLOBAL ] VARIABLES LIKE '.....'; -- 可以通过LIKE模糊匹配方式查找变量
SELECT @@[SESSION | GLOBAL] 系统变量名;         -- 查看指定变量的值
```

- 设置系统变量

```
SET [ SESSION | GLOBAL ] 系统变量名 = 值 ;
SET @@[SESSION | GLOBAL]系统变量名 = 值 ;
```

用户自定义变量

- 变量

用户定义变量 是用户根据需要自己定义的变量，用户变量不用提前声明，在用的时候直接用“@变量名”使用就可以。其作用域为当前连接。

- 赋值

```
SET @var_name = expr [, @var_name = expr] ... ;
SET @var_name := expr [, @var_name := expr] ... ;

SELECT @var_name := expr [, @var_name := expr] ... ;
SELECT 字段名 INTO @var_name FROM 表名;
```

- 使用

```
SELECT @var_name ;
```

注意:
用户定义的变量无需对其进行声明或初始化，只不过获取到的值为NULL。

局部变量

局部变量 是如果需要定义的在局部生效的变量，访问之前，需要DECLARE声明。可用作存储过程内的局部变量和输入参数，局部变量的范围是在其内声明的BEGIN ... END块。

- 声明

```
DECLARE 变量名 变量类型 [DEFAULT ...] ;
```

变量类型就是数据库字段类型：INT、BIGINT、CHAR、VARCHAR、DATE、TIME等

- 赋值

```
SET 变量名 = 值 ;
SET 变量名 := 值 ;
SELECT 字段名 INTO 变量名 FROM 表名 ... ;
```


If

2025年12月17日 21:11

- if

语法:

```
IF 条件1 THEN
    .....
ELSEIF 条件2 THEN      -- 可选
    .....
ELSE                    -- 可选
    .....
END IF;
```


参数

2025年12月17日 21:13

- 参数

类型	含义	备注
IN	该类参数作为输入，也就是需要调用时传入值	默认
OUT	该类参数作为输出，也就是该参数可以作为返回值	
INOUT	既可以作为输入参数，也可以作为输出参数	

用法:

```
CREATE PROCEDURE 存储过程名称 ([ IN/OUT/INOUT 参数名 参数类型 ])
BEGIN
    -- SQL语句
END ;
```

Case

2025年12月17日 21:48

- case

- 语法一

```
CASE case_value
    WHEN when_value1 THEN statement_list1
    [ WHEN when_value2 THEN statement_list2 ] ...
    [ ELSE statement_list ]
END CASE;
```

- 语法二

```
CASE
    WHEN search_condition1 THEN statement_list1
    [ WHEN search_condition2 THEN statement_list2 ] ...
    [ ELSE statement_list ]
END CASE;
```

```
83
84 CREATE PROCEDURE p6(in month int)
85 BEGIN
86
87 DECLARE result VARCHAR(50);
88
89 CASE
90     WHEN month >= 1 and month <= 3 THEN
91         set result := '第一季度';
92     WHEN month >= 4 and month <= 6 THEN
93         set result := '第二季度';
94     WHEN month >= 10 and month <= 12 THEN
95         set result := '第三季度';
96     ELSE
97         set result := '非法参数';
98 END CASE;
99
100 SELECT concat('您输入的月份为:', month, ',所属的季度为', result);
101
102 END;
103
104 DROP PROCEDURE p6
105
106 CALL p6(10);
107
```

While

2025年12月17日 22:02

- while

while 循环是有条件的循环控制语句。满足条件后，再执行循环体中的SQL语句。具体语法为：

#先判定条件，如果条件为true，则执行逻辑，否则，不执行逻辑

```
WHILE 条件 DO
```

```
    SQL逻辑...
```

```
END WHILE;
```

Repeat

2025年12月17日 22:35

- repeat

repeat是有条件的循环控制语句, 当满足条件的时候退出循环。具体语法为:

#先执行一次逻辑, 然后判定逻辑是否满足, 如果满足, 则退出。如果不满足, 则继续下一次循环

REPEAT

SQL 逻辑...

UNTIL 条件

END REPEAT;

Loop

2025年12月17日 22:38

- loop

LOOP 实现简单的循环，如果不在SQL逻辑中增加退出循环的条件，可以用其来实现简单的死循环。LOOP可以配合一下两个语句使用：

- LEAVE：配合循环使用，退出循环。
- ITERATE：必须用在循环中，作用是跳过当前循环剩下的语句，直接进入下一次循环。

```
[begin_label:] LOOP
    SQL逻辑...
END LOOP [end_label];
```

函数

2025年12月17日 22:38

存储函数是有返回值的存储过程，存储函数的参数只能是IN类型的。具体语法如下：

```
CREATE FUNCTION 存储函数名称 ([ 参数列表 ])
RETURNS type [characteristic ...]
BEGIN
    -- SQL语句
    RETURN ...;
END ;
```

characteristic说明：

- DETERMINISTIC：相同的输入参数总是产生相同的结果
- NO SQL：不包含 SQL 语句。
- READS SQL DATA：包含读取数据的语句，但不包含写入数据的语句。

游标

2025年12月22日 16:54

- 游标

游标（CURSOR）是用来存储查询结果集的数据类型，在存储过程和函数中可以使用游标对结果集进行循环的处理。游标的使用包括游标的声明、OPEN、FETCH 和 CLOSE，其语法分别如下。

➤ 声明游标

```
DECLARE 游标名称 CURSOR FOR 查询语句；
```

➤ 打开游标

```
OPEN 游标名称；
```

➤ 获取游标记录

```
FETCH 游标名称 INTO 变量[, 变量 ]；
```

➤ 关闭游标

```
CLOSE 游标名称；
```

条件处理程序

2025年12月22日 17:54

- 条件处理程序

条件处理程序（Handler）可以用来定义在流程控制结构执行过程中遇到问题时相应的处理步骤。具体语法为：

```
DECLARE handler_action HANDLER FOR condition_value [, condition_value] ... statement ;
```

handler_action

CONTINUE: 继续执行当前程序

EXIT: 终止执行当前程序

condition_value

SQLSTATE sqlstate_value: 状态码，如 02000

SQLWARNING: 所有以01开头的SQLSTATE代码的简写

NOT FOUND: 所有以02开头的SQLSTATE代码的简写

SQLEXCEPTION: 所有没有被SQLWARNING 或 NOT FOUND捕获的SQLSTATE代码的简写

```
CREATE PROCEDURE p11(in uid int)
BEGIN
    DECLARE ename VARCHAR(100);
    DECLARE salary DECIMAL(10,2);
    DECLARE e_cursor CURSOR FOR SELECT emp name, salary FROM employees where emp_id < uid;
    DECLARE EXIT HANDLER FOR NOT FOUND CLOSE e_cursor;

    CREATE TABLE if not EXISTS tb_emp_info(
        id int PRIMARY KEY AUTO_INCREMENT,
        name VARCHAR(100),
        salary DECIMAL(10,2)
    );

    OPEN e_cursor;

    while true do
        FETCH e_cursor into ename, salary;
        INSERT INTO tb_emp_info VALUES(null, ename, salary);
    end while;

    CLOSE e_cursor;
```


介绍

2025年12月22日 16:54

- 介绍

触发器是与表有关的数据库对象，指在 insert/update/delete 之前或之后，触发并执行触发器中定义的SQL语句集合。触发器的这种特性可以协助应用在数据库端确保数据的完整性，日志记录，数据校验等操作。

使用别名 OLD 和 NEW 来引用触发器中发生变化的记录内容，这与其他数据库是相似的。现在触发器还支持行级触发，不支持语句级触发。

id	name
1	javaEE
2	Spring
3	MySQL
4	Cloud

触发器类型	NEW 和 OLD
INSERT 型触发器	NEW 表示将要或者已经新增的数据
UPDATE 型触发器	OLD 表示修改之前的数据, NEW 表示将要或已经修改后的数据
DELETE型触发器	OLD 表示将要或者已经删除的数据

行级触发：当执行了UPDATE触发器它影响了五行那么这个触发器会被触发五次这个就成为行级触发。

语句级触发：比如执行了这个UPDATE不管它影响多少行，但是它只触发一次，这就要语句级触发器

语法

2025年12月22日 23:53

● 语法

➤ 创建

```
CREATE TRIGGER trigger_name
BEFORE/AFTER INSERT/UPDATE/DELETE
ON tbl_name FOR EACH ROW -- 行级触发器
BEGIN
    trigger_stmt;
END;
```

➤ 查看

```
SHOW TRIGGERS;
```

➤ 删除

```
DROP TRIGGER [schema_name.]trigger_name; -- 如果没有指定 schema_name, 默认为当前数据库。
```

```
CREATE TRIGGER tb_emp_insert_trigger
AFTER INSERT ON employees FOR EACH ROW
] BEGIN
    INSERT INTO emp_logs(operation, operate_time, operate_id, operate_params)
] VALUES (
    'insert',
    NOW(),
    NEW.emp_id,
] CONCAT(
    '插入的数据内容为: emp_id=', NEW.emp_id,
    ', name=', NEW.emp_name, |
    ', salary=', NEW.salary,
    ', dept_id=', NEW.dept_id,
    ', hire_date=', DATE_FORMAT(NEW.hire_date, '%Y-%m-%d %H:%i:%s')
- )
- );
END
```

```
SHOW TRIGGERS
```

```
DROP TRIGGER test.tb_emp_insert_trigger
```

```
INSERT INTO employees(emp_name, salary, dept_id, hire_date)VALUES('小丁', 9999.00, 1, now())
```

注意：如果原触发器内的其他语句有语法错误但不影响触发器的创建，在修复错误后需要重新创建触发器，不然执行的还是有语法错误那个触发器指令。